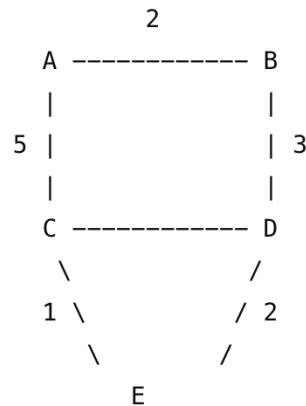


Exercise 2: Link State and Distance Vector Routing

Name: Thanapong Khongkasem **Student ID:** 6710405371 **Date:** 20/9/2026

Learning objectives: Apply Dijkstra and Bellman-Ford; construct forwarding tables; analyze convergence, failures, and count-to-infinity.

Network topology:



Part 1 — Link State Routing

Question 1 — Dijkstra's Algorithm

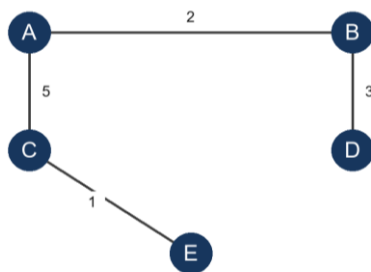
Starting at router A, execute Dijkstra's algorithm. Show every update and the node confirmed at each step.

Step	Confirmed Node	D(B), Prev.	D(C), Prev.	D(D), Prev.	D(E), Prev.
0	A	2, A	5, A	∞ , -	∞ , -
1	B	2, A	5, A	5, B	∞ , -
2	C	2, A	5, A	5, B	6, C
3	D	2, A	5, A	5, B	6, C
4	E	2, A	5, A	5, B	6, C

Question 2 — Shortest-Path Tree

Draw the shortest-path tree rooted at A, then list the shortest path and total cost.

SPT sketch:



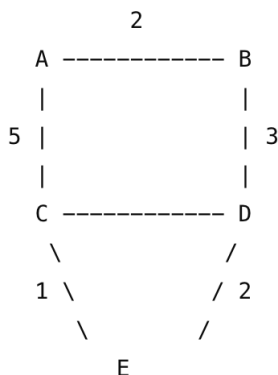
Destination	Shortest Path	Total Cost
B	A -> B	2
C	A -> C	5
D	A -> B -> D	5
E	A -> C -> E	6

Question 3 — Forwarding Table

Destination	Shortest Cost	Next Hop	Complete Path
B	2	B	A -> B
C	5	C	A -> C
D	5	B	A -> B -> D
E	6	C	A -> C -> E

Part 2 — Distance Vector Routing

Consider the same network. Initially, each router knows only the cost to its directly connected neighbors.



Question 4 — Bellman-Ford Update

Router A receives the following distance vectors from its neighbors:

From B:

A=2, B=0, C=4, D=3, E=5;

From C:

A=5, B=4, C=0, D=1, E=1.

Use the Bellman-Ford equation. Calculate router A's new distance to:

Show all calculations, including the neighbor through which the minimum-cost route is obtained.

Destination	Via B calculation	Via C calculation	Minimum Cost	Next Hop
B	$2 + 0 = 2$	$5 + 4 = 9$	2	B
C	$2 + 4 = 6$	$5 + 0 = 5$	5	C
D	$2 + 3 = 5$	$5 + 1 = 6$	5	B
E	$2 + 5 = 7$	$5 + 1 = 6$	6	C

Question 5 — Distance Vector Table

After receiving the distance vectors, construct router A's updated routing table.

Destination	Next Hop	Distance	Full Path
B	B	2	A -> B
C	C	5	A -> C
D	B	5	A -> B -> D
E	C	6	A -> C -> E

Part 3 — Routing Convergence

Question 6 — Link Failure

The C–D link fails. Explain separately what happens with Link State and Distance Vector routing.

Link State routing.

Your answer should describe:

- Which router(s) detect the failure.
- What information is flooded through the network.
- How routers update their link-state databases.
- How routers calculate new shortest paths.

Distance Vector routing.

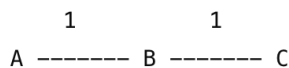
Your answer should describe:

- How routers learn about the failure.
- How updated distance vectors propagate.
- Why routers may temporarily have incorrect routing information.
- How routing eventually converges.

Link State	Distance Vector
C และ D ตรวจพบความเสียหายแล้ว flood สถานะลิงก์	C และ D แจ้งค่าใหม่แก่เพื่อนบ้าน
ทุกตัวปรับ topology database	แต่ละตัวปรับจาก vector ที่ได้รับ
ทุกตัวรัน Dijkstra ใหม่	แต่ละตัวรัน Bellman-Ford ซ้ำ
โดยทั่วไป converge เร็วกว่า	อาจมีข้อมูลเก่าและ loop ชั่วคราว
C และ D ตรวจพบความเสียหายแล้ว flood สถานะลิงก์	C และ D แจ้งค่าใหม่แก่เพื่อนบ้าน

Part 4 — Count-to-Infinity

Consider



Initially A reaches C through B at cost 2. The B–C link fails, but B receives stale information from A saying C is reachable at cost 2.

Question 7

1. Why can this create a routing loop?
A เชื่อว่าต้องไป C ผ่าน B ขณะที่ B เชื่อข้อมูลเก่าของ A และคิดว่าสามารถไป C ผ่าน A ได้ ดังนั้นแพ็กเก็ตจะหมุนวนระหว่าง A กับ B
2. Show how the estimated distance to C can increase over successive iterations.

รอบ	ค่าที่ A ประมาณไป C	ค่าที่ B ประมาณไป C	เหตุการณ์
ก่อนเสีย	2 ผ่าน B	1 โดยตรง	ลิงก์ B-C ยังทำงาน
1	2 ซึ่งเป็นค่าเก่า	3 ผ่าน A	B คำนวณ 1 + 2
2	4 ผ่าน B	3	A คำนวณ 1 + 3
3	4	5 ผ่าน A	B คำนวณ 1 + 4

รอบ	ค่าที่ A ประมาณไป C	ค่าที่ B ประมาณไป C	เหตุการณ์
4	6 ผ่าน B	5	A คำนวณ 1 + 5

3. What is count-to-infinity?

เป็นเหตุการณ์ที่เราเตอร์ค่อย ๆ เพิ่ม **metric** ของปลายทางที่ไม่สามารถเข้าถึงได้ จนถึงค่าที่โปรโตคอลกำหนดว่าเป็น **infinity** ตัวอย่างเช่น RIP กำหนดให้ **hop count 16** หมายถึง **unreachable**

4. Why does it occur in Distance Vector routing?

แต่ละเราเตอร์ไม่รู้ **topology** ทั้งหมด จึงไม่รู้ว่าจะเส้นทางที่เพื่อนบ้านประกาศกลับมานั้นอาศัยเส้นทางผ่านตัวมันเอง

5. Name a mitigation approach.

ใช้ **split horizon with poison reverse** คือ ถ้า A เรียนรู้เส้นทางไป C จาก B แล้ว A จะประกาศกลับไปยัง B ว่าเส้นทางนั้นมีค่า **infinity** ทำให้ B ไม่เลือกเส้นทางย้อนกลับผ่าน A วิธีเสริมอื่น ได้แก่ **route poisoning, triggered updates** และ **hold-down timers**

Part 5 — Compare LS and DV

Question 8

Characteristic	Link State	Distance Vector
Basic algorithm	สร้างแผนที่แล้วหา shortest path	เลือกค่าต่ำสุดจากข้อมูลเพื่อนบ้าน
Information maintained	Topology และสถานะลิงก์	ระยะทางและ next hop
Information exchanged	Link State Advertisement	Distance vector หรือ routing table
Algorithm	Dijkstra	Bellman-Ford
Topology knowledge	รู้ topology ภายใน area	ไม่รู้ topology ทั้งหมด
Routing updates	Flood เมื่อสถานะเปลี่ยนและ refresh เป็นระยะ	ส่งให้เพื่อนบ้านเป็นระยะหรือเมื่อค่าเปลี่ยน
Convergence	โดยทั่วไปเร็วกว่า	โดยทั่วไปช้ากว่า
Routing loops	มีโอกาสน้อยหลังฐานข้อมูลตรงกัน	อาจเกิดระหว่าง convergence
Count-to-infinity	ไม่เกิดในรูปแบบของ DV	อาจเกิด
Example protocol	OSPF และ IS-IS	RIP

Part 6 — Design Question

Question 9

You are designing a routing system for an enterprise with 500 routers requiring fast convergence, accurate topology knowledge, efficient routing, and hierarchical design. Choose LS or DV and justify your choice with at least three technical arguments. Your answer should discuss at least three advantages and/or disadvantages of your chosen approach.

ควรเลือก Link State เช่น OSPF หรือ IS-IS ด้วยเหตุผลต่อไปนี้

1. **Convergence** เร็ว เมื่อมีลิงก์เปลี่ยนแปลง ระบบจะ **flood** สถานะใหม่และคำนวณเส้นทางอีกครั้ง ไม่ต้องรอข่าวแพร่ที่ละ **hop** หลายรอบ
2. มีความรู้ **topology** ที่ถูกต้อง แต่ละเราเตอร์มี **link-state database** จึงคำนวณจากสภาพเครือข่ายจริงและลดความเสี่ยงจากข้อมูล **stale**
3. รองรับการออกแบบแบบลำดับชั้น **OSPF** แบ่งเครือข่ายเป็น **areas** และ **IS-IS** แบ่งเป็น **levels** ช่วยจำกัดขนาดฐานข้อมูลและขอบเขตการ **flood**
4. รองรับเครือข่ายขนาดใหญ่ได้ดีกว่า และไม่มีปัญหา **count-to-infinity** แบบ **Distance Vector**

ข้อเสียคือใช้หน่วยความจำและกำลังประมวลผลมากกว่า อีกทั้งการออกแบบ **area** หรือ **level** ต้องมีการวางแผน อย่างไรก็ตาม สำหรับ **500 routers**

ข้อดีด้านความเร็ว ความแม่นยำ และ **scalability** มีน้ำหนักมากกว่า

Challenge — Algorithmic Analysis

Question 10

For N routers and E links, compare LS and DV in information maintained, communication overhead, computation, convergence, and scalability. Explain why large-scale intra-domain routing generally favors Link State.

เรื่อง	Link State	Distance Vector
ข้อมูลที่เก็บ	Topology graph ขนาด $O(N + E)$ ต่อเราเตอร์	Distance vector ขนาด $O(N)$ ต่อเราเตอร์
การสื่อสาร	การเปลี่ยนแปลงหนึ่งรายการถูก flood ผ่านเครือข่ายประมาณ $O(E)$ transmissions	แต่ละรอบส่ง vector ขนาด $O(N)$ ผ่านลิงก์ รวมประมาณ $O(NE)$ entries ต่อรอบ
การคำนวณ	Dijkstra ใช้ $O((N + E) \log N)$ ด้วย priority queue หรือ $O(N^2)$ แบบพื้นฐาน	เราเตอร์หนึ่งตัวใช้ประมาณ $O(N \times \text{degree})$ ต่อรอบ
Convergence	เร็วและคาดการณ์ได้กว่า	ต้องอัปเดตหลายรอบและอาจเกิด count-to-infinity
Scalability	ดีเมื่อแบ่งเป็น area หรือ level	ลดลงเมื่อ vector ใหญ่และมีหลายรอบการอัปเดต

เครือข่าย **intra-domain** ขนาดใหญ่มักเลือก **Link State** เพราะการเปลี่ยนแปลงแพร่ได้รวดเร็ว เราเตอร์มีมุมมอง **topology** ที่สอดคล้องกัน มีโอกาสเกิด **routing loop** น้อยกว่า และสามารถแบ่งเครือข่ายเป็น **area** หรือ **level** เพื่อควบคุม **overhead** ได้